Journal Extension: Complete Plan

Optimized Bus Route Finder for Educational Institutes

From Conference Paper to AI-Powered Intelligent Campus Transit System

1. How Cited Papers Used Our Conference Paper

Four papers have cited the conference paper (ICCCNT 2024). The table below summarises how each paper referenced it, what their own research contributed, and how they extended or applied the Haversine formula differently.

Paper / Venue	How They Cited C1	What They Actually Built	How They Extended Beyond Haversine
Veres (2025) Logistics, MDPI	Ref [12]. Discussed in body text: described C1 as using Haversine as a "backend calculation mechanism, not a benchmarked algorithm."	Compared 5 distance estimation methods (Haversine with circuity, linear/polynomial regression, ANN) across 5,000 Hungarian postal locations. ANN produced the most map-consistent routes.	Haversine is a baseline to beat. ML (ANN) corrects straight-line error by learning road sinuosity patterns. Proved Haversine alone is insufficient for route planning.
Arockiam et al. (2025) iConSCEPT, IEEE	Ref [2]. One-line mention in literature survey: "Senthilkumar et al. have used haversine distance to find the best Bus route aiming at Educational Institutes."	Used Haversine distance between consecutive GPS packets to filter GPS drift in emergency vehicle tracking. Different thresholds for stationary vs moving vehicles. Removed 54% of drifted location points.	Repurposed Haversine for data quality filtering, not location finding. Same formula, completely different application (signal processing rather than proximity search).
Baskoro et al. (2026) IJRCS	Ref [30]. Reference list only. No discussion in body text.	Trained KNN regression to predict reachable distance from remaining motorcycle fuel. Then used Haversine to filter petrol stations within predicted range. Built Android app. MAE below 0.21 km, R-squared above 0.80.	Two-step pipeline: ML predicts a range first, then Haversine geofences candidates within that range. Haversine is a filtering step after ML, not the primary algorithm.
Khairnar & Sonkamble (2026) JIDMIS	Ref [99] out of 100+ references. Reference list only. No in-text discussion.	Built a 3-stage intelligent traffic routing framework: (1) feature extraction using Mutual Information and Random Forest, (2) traffic prediction using hybrid SARIMAX + XGBoost, (3) dynamic graph routing with Dijkstra on time-varying edge weights. Achieved 18.3% travel time reduction over static Dijkstra on London traffic data.	Most sophisticated of the four. Full ML prediction pipeline coupled with graph-based routing. C1 was cited only as bibliography padding.

Table 1. Analysis of papers that cited the conference paper.

Key takeaway: no citing paper extended the campus bus route finding problem itself. All four used the C1 paper as a reference to the Haversine formula being applied in a domain-specific context. Veres explicitly noted that C1 uses Haversine as an unbenmarked backend utility. This criticism directly motivates the proposed journal extension, which benchmarks the Haversine-only approach against an intelligent agent-based alternative.

2. Literature Review: Paper-by-Paper Research Gaps

A structured review of 34 papers across seven thematic domains was conducted. The following table presents each paper, its methodology, and the specific research gap identified relative to the proposed intelligent campus bus navigation system.

Paper / Venue	Method	Key Contribution	Gap (vs. Proposed System)
Senthilkumar et al. (2024) ICCCNT IEEE	Haversine + Google Maps	Campus stop finder within 2km	No real travel time, no natural language query, no intelligent ranking. Tested with only 1 location.
Noor et al. (2021) Sustainability, MDPI	ANN + SVM regression	Campus shuttle travel time prediction	No stop proximity search, no conversational interface, no driver accountability display.
Aggarwal et al. (2024) ICRASET IEEE	ESP32 GPS + IoT + Cloud	SRM Chennai campus shuttle tracking	No nearest-stop search, no natural language queries, no trip planning or route comparison.
Thothong et al. (2025) IEEE, KMITL	ESP32 + 4G + Leaflet maps	Campus shuttle tram live tracking	Pure hardware telemetry. No intelligent recommendation, no schedule queries, no chat interface.
Shaikh et al. (2024) IJARSCT	GPS + RFID + GSM	College bus with RFID attendance	Safety-focused tracking. No route optimization, no query interface, no travel time estimation.
Jabbar & Pinandito (2023) Univ. Brawijaya	ACO vs Dijkstra	Transit route recommendation	Static graph routing. No campus context, no real-time traffic, no natural language interface.
Dogra et al. (2025) IEEE	Hybrid Genetic Algorithm	Dynamic departure time optimization	Urban-scale focus. No user-facing stop search or conversational querying capability.
Li & Kim (2024) CMC	Multi-obj. heuristic A*	Demand-responsive transit routing	Assumes on-demand dispatch. Not applicable to fixed-schedule institutional bus systems.
Nair et al. (2022) J. Adv. Transportation	Gradient Boosting + SVM	Bus GPS to traffic travel time mapping	Traffic sensing focus. No user-facing navigation, stop finder, or query interface.
Varma et al. (2024) IEEE	Kalman filter + SVR	Bus ETA prediction with GPS smoothing	Urban city buses. No institutional schedule context, no driver contacts, no conversational AI.
Sharon Drisilda (2025) IJSREM	SARIMAX time series	Hourly ridership forecasting for BMTC	Demand prediction only. No route finding, no proximity search, no natural language interface.
Nampalli et al. (2025) IEEE	LSTM + Bluetooth sensors	Intersection travel time prediction	Requires sensor infrastructure. No campus bus context, no user-facing navigation.
Cheng & He (2024) ERA, AIMS	NSGA-II Pareto optimization	Bus timetable optimization (Lhasa)	Operator-side planning tool. No commuter interface, no stop search, no conversational query.
Ahern et al. (2022) Trans. Res. Part B	Simulated Annealing	Route design + frequency setting	Theoretical metropolitan modeling. No user-facing application, no real-time features.
Khairnar & Sonkamble (2026) JIDMIS	SARIMAX + XGBoost + TSBG	Dynamic graph routing with ML prediction	Urban traffic scale. No campus bus schedules, no stop proximity, no natural language queries.
Veres (2025) Logistics, MDPI	ANN vs polynomial regression	Distance estimation without maps	Logistics delivery context. No transit scheduling, no user-facing navigation or chat.
Arockiam et al. (2025) iConSCEPT IEEE	Haversine packet filtering	GPS drift removal for emergency vehicles	Domain-specific signal processing. No transit routing, no schedule queries, no campus context.
Baskoro et al. (2026) IJRCS	KNN + Haversine filtering	Fuel-aware gas station recommendation	Motorcycle refueling context. No public transit, no scheduling, no conversational interface.
Wang et al. (2019) FGCS, Elsevier	Cloud-terminal GIS	Spatiotemporal big data architecture	Enterprise GIS infrastructure. Not tailored for lightweight campus transit applications.
Xie et al. (2022) PVLDB, Alibaba	Ganos cloud-native spatial DB	Trajectory and spatial database engine	Database layer only. No user-facing routing, no proximity filters, no conversational AI.

Paper / Venue	Method	Key Contribution	Gap (vs. Proposed System)
Bonam et al. (2025) IEEE	Cloud geospatial ML survey	Meta-analysis of 58 geospatial systems	Architectural guidelines only. No concrete campus transit implementation.

Table 2. Literature review with identified research gaps (representative selection of 21 papers shown).

3. Research Gaps the Proposed System Addresses

Five core gaps emerge from the literature. The table below maps each gap to the specific component in the proposed system that resolves it.

Research Gap	Evidence from Literature	How Our System Solves It
Gap 1: Proximity is used as a proxy for best route	Every campus bus paper (including C1) assumes the nearest stop is the best option. No system checks whether the nearest stop actually results in the shortest total trip (walking + waiting + bus travel).	The AI agent computes total trip time for each candidate stop using the Directions API (walking time to stop + bus travel time to campus) and recommends the genuinely fastest option, not just the nearest.
Gap 2: No natural language query interface for campus transit	All reviewed campus bus systems use dropdown menus, coordinate inputs, or map-only interfaces. No system allows users to ask questions in plain English like "any bus from Chromepet in the morning?"	RAG pipeline (ChromaDB + Gemini) indexes all route, schedule, stop, and driver data. Users type questions naturally. The system retrieves relevant data and generates accurate answers grounded in the verified database.
Gap 3: Cross-route queries are impossible	Existing apps show one route at a time. A student asking "which route goes through both Velachery and Adyar?" or "what are all my options from T. Nagar?" would have to open every route manually and compare.	RAG searches across ALL routes simultaneously. The agent retrieves and compares information spanning multiple routes, stops, and time slots in a single response.
Gap 4: No goal-directed trip planning	No campus bus system accepts a goal like "reach campus by 9 AM" and works backwards to recommend the best departure. Students must manually calculate which bus arrives in time.	The AI agent accepts time-constrained goals, filters schedules for buses arriving before the deadline, computes walking and travel times, and recommends the latest possible departure that still meets the constraint.
Gap 5: Haversine is unvalidated as a recommendation method	Veres (2025) explicitly noted that C1 uses Haversine as a "backend calculation mechanism, not a benchmarked algorithm." No study has measured whether Haversine proximity actually correlates with best route choice.	The journal paper directly benchmarks Haversine-only recommendations against agent recommendations across multiple test locations, measuring total trip time difference. This provides the first empirical evaluation of proximity-based campus bus recommendation.

Table 3. Research gaps and corresponding solutions in the proposed system.

4. How the Final Application Looks

The application consists of two side-by-side sections and a shared map at the bottom. The left side preserves the original conference paper interface without any changes. The right side adds the new AI chat assistant.

4.1 Left Side: Original Interface (Unchanged)

The left panel contains every feature from the published conference paper: a time slot selector (morning, afternoon, evening), a route dropdown menu, a search button that loads the stops and timings table, driver and in-charge contact display, and the nearest stop finder with latitude/longitude input fields. Students familiar with the existing system continue using it exactly as before.

4.2 Right Side: AI Chat Assistant (New)

The right panel is a conversational chat interface. The student types a question in plain English. The AI agent processes the question, calls the appropriate tools (Haversine, RAG retrieval, Google Directions API), and responds with a grounded, accurate answer. The chat supports follow-up questions and maintains conversation context across multiple exchanges.

4.3 Example Interactions

Query Type	Example User Input	What the System Does
Simple area-based lookup	**"Any bus from Chromepet in the morning?"**	The agent queries RAG across all route data for stops matching Chromepet in the morning slot. Responds with matching routes, stop names, and departure times. No coordinates needed.
Cross-route comparison	**"Which route goes through both Velachery and Adyar?"**	The agent searches all route documents for stops near both areas. Returns routes serving both, with stop names and timings. Impossible with the dropdown interface.
Goal-directed trip planning	**"I need to reach campus by 9 AM. I am near Vadapalani."**	The agent runs a multi-step plan: (1) Haversine finds nearby stops, (2) RAG retrieves departure times for each, (3) Directions API computes walking time to each stop and bus travel time to campus, (4) Agent computes total trip time for each option and recommends the fastest, explaining the tradeoff between walking distance and bus speed.
Follow-up question	**"What if I leave 20 minutes later?"**	The agent recalls the previous plan, re-checks which buses are still available, recalculates, and provides an updated recommendation. Conversational memory allows multi-turn planning.
Accountability query	**"My bus did not come. Who do I contact for Route 12?"**	RAG retrieves driver and in-charge contact details for Route 12. No dropdown navigation needed.
Exploratory discovery	**"What are all my options from T. Nagar?"**	RAG searches across all routes for stops near T. Nagar across all time slots. Returns a complete list of routes, stops, and timings. One question replaces manually checking 30 routes.

Table 4. Example interactions demonstrating the AI chat assistant capabilities.

5. How the Journal System Improves Over the Conference System

Dimension	Conference Paper (ICCCNT 2024)	Journal Extension (Proposed)
User input method	Exact latitude and longitude coordinates	Plain English text (area names, landmarks, questions)
Query scope	One route at a time via dropdown	Searches across all routes simultaneously
What it recommends	Nearest stop by straight-line distance	Fastest total trip (walk + wait + bus travel)
Travel time awareness	None. Shows only schedule times.	Real-time travel duration from Google Directions API
Natural language queries	Not supported	Full conversational interface via RAG + Gemini
Cross-route comparison	Not possible. Must check routes one by one.	Agent compares multiple routes in a single response
Goal-directed planning	Not supported	Accepts constraints like "reach by 9 AM" and plans backwards
Follow-up questions	Not supported	Conversational memory maintains context across exchanges
Driver contact access	Must select the exact route from dropdown first	Ask in chat: "Who drives Route 12?" and get instant answer
Intelligence	Haversine radius filter (textbook formula)	AI agent with multi-step reasoning and tool use
Number of test locations	1	20 to 30 locations across Chennai (planned evaluation)
Algorithmic evaluation	None (Veres 2025 noted this as a weakness)	Ablation study comparing Haversine-only vs agent recommendations

Table 5. Feature comparison between the conference system and the proposed journal system.

The fundamental improvement is conceptual: the conference system answers "which stop is closest?" while the journal system answers "which bus should I take?" The first question has a simple geometric answer (Haversine). The second requires reasoning across schedules, travel times, walking distances, and user constraints. That reasoning is what the AI agent provides.

6. Journal Approval Strategy

6.1 Research Contribution (Why a Journal Would Accept This)

Journal reviewers evaluate papers on novelty, rigor, and significance. The proposed paper delivers on all three:

- **Novelty:** First study to demonstrate that proximity-based campus bus recommendation is suboptimal, and first application of an LLM-based agentic system with RAG to institutional transit navigation.
- **Rigor:** Empirical comparison between Haversine-only and agent-based recommendations across 20 to 30 test locations, with quantitative trip time measurements and a standardized user study.
- **Significance:** Directly applicable to any educational institution with bus transportation. The system is deployed on real VIT Chennai route data.

6.2 Proposed Paper Structure (15+ Pages)

Section	Content
Introduction	Campus transit problem, motivation, research question
Literature Review	34 papers across 7 domains (already completed)
Motivation Study	Quantify how often Haversine picks suboptimal routes using VIT data
System Architecture	Agent design, tool definitions, RAG pipeline, data flow
Implementation	React + FastAPI + LangGraph + ChromaDB + Gemini + Directions API
Evaluation Setup	20 to 30 test locations, 4 ablation configurations, user study design
Results	Trip time comparison, RAG retrieval accuracy, user study (SUS scores)
Discussion	When the agent helps most, limitations, generalizability
Conclusion	Summary of contributions, future work

Table 6. Proposed journal paper structure.

6.3 Evaluation Plan (What Reviewers Will Look For)

The evaluation has three parts:

- **Ablation study (4 configurations):** C1 = Haversine only (conference baseline). C2 = Haversine + Directions API (real travel times, no agent). C3 = Full agent with RAG + trip planning. C4 = Full agent + conversational follow-ups. Comparing C1 vs C3 directly measures the benefit of the agent.
- **Quantitative metrics:** Total trip time difference (minutes saved by agent vs Haversine-only). RAG retrieval precision (proportion of correctly retrieved route chunks). Response latency (seconds from query to answer).
- **User study:** 25+ VIT Chennai students who use the institutional bus service. Each participant uses both the original system and the agent system in counterbalanced order. System Usability Scale (SUS) questionnaire after each. Paired t-test for statistical significance.

6.4 Target Journals

Journal	Publisher	IF	Rationale
Sustainability	MDPI	3.9	Direct precedent: Noor et al. (2021) campus shuttle ML paper published here
Expert Systems with Applications	Elsevier	8.5	RAG + AI agent decision support system fits scope directly
IEEE Trans. on ITS	IEEE	8.5	Flagship intelligent transportation journal
Applied Sciences	MDPI	2.7	Broad applied CS and transport; accessible review timeline
J. of Advanced Transportation	Wiley	2.3	Precedent: Nair et al. (2022) ML bus paper on Indian roads

Table 7. Target journals ranked by scope alignment.

6.5 What Makes This Different from Adding Features

The journal extension is not a feature list added on top of the conference paper. It is structured around a testable research question: does Haversine proximity actually recommend the best bus route? The answer (demonstrated through evaluation) is no. The AI agent is the proposed method that answers the question better. The RAG pipeline enables natural language interaction that makes the system usable without coordinate inputs. Every component serves the central research question, and every claim is backed by measurable evaluation. This structure is what distinguishes a journal contribution from an engineering project.